

REPUBLIQUE DU SENEGAL



Un peuple _ Un but _ Une foi

Ministère de l'Enseignement Supérieur,
de la Recherche et de l'Innovation

Université Assane Seck de Ziguinchor
Faculté des Sciences et Technologies
Département d'Informatique

Mémoire de Master 1 – Réseaux Avancés

Thème :

Architecture SDN pour la Gestion Dynamique du Trafic et la Sécurité Réseau

Auteure :
Aminata Thior

Encadrant :
Pr. Malick Ndoeye

Année universitaire 2024–2025

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet.

En premier lieu, je remercie sincèrement mon encadrant, **Professeur Malick Ndoye**, pour sa disponibilité, ses conseils avisés et son soutien tout au long de ce travail. Sa rigueur scientifique et ses orientations ont été déterminantes dans la réussite de ce projet.

Je remercie également l'ensemble du corps enseignant du **Département d'Informatique** de l'Université Assane Seck de Ziguinchor pour la qualité de la formation dispensée tout au long de mon cursus de Master.

Je tiens à exprimer ma reconnaissance à ma **famille**, pour leur soutien indéfectible, leur patience et leurs encouragements constants durant toutes mes années d'études.

Enfin, je remercie mes camarades de promotion pour les échanges enrichissants et l'entraide tout au long de cette année universitaire.

Résumé

Les réseaux traditionnels souffrent de rigidité et d'un manque de visibilité globale, rendant difficile la gestion dynamique du trafic et la réponse rapide aux menaces de sécurité. Le paradigme SDN (Software-Defined Networking) propose une approche innovante en séparant le plan de contrôle du plan de données, offrant ainsi une vue centralisée et programmable du réseau.

Ce projet présente la conception et le déploiement d'une architecture SDN complète basée sur le contrôleur Ryu et l'émulateur Mininet. Quatre modules ont été développés : un module de surveillance du trafic en temps réel, un module de routage dynamique basé sur les métriques de performance, un module QoS pour la priorisation des flux, et un module de sécurité intégrant la détection DDoS par entropie de Shannon, la mitigation automatique graduée et un pare-feu stateful.

Les évaluations sur une topologie arbre (16 hôtes) montrent un débit TCP de 23,5 Gbps, une latence de 0,134 ms, une détection DDoS en 5 secondes et une réduction de la latence de 52% par rapport à un réseau traditionnel.

Mots-clés : SDN, OpenFlow, Ryu, Mininet, DDoS, QoS, Sécurité réseau, Gestion du trafic.

Abstract

Traditional networks suffer from rigidity and lack of global visibility, making dynamic traffic management and rapid response to security threats difficult. The SDN paradigm proposes an innovative approach by separating the control plane from the data plane, offering a centralized and programmable view of the network.

This project presents the design and deployment of a complete SDN architecture based on the Ryu controller and Mininet. Four modules were developed : real-time traffic monitoring, dynamic routing, QoS flow prioritization, and a security module with DDoS detection via Shannon entropy, automatic mitigation and a stateful firewall.

Evaluations show 23.5 Gbps TCP throughput, 0.134 ms latency, DDoS detection in 5 seconds and 52% latency reduction.

Keywords : SDN, OpenFlow, Ryu, Mininet, DDoS, QoS, Network Security, Traffic Management.

Table des matières

Remerciements	1
Résumé	2
Abstract	3
Liste des figures	8
Liste des tableaux	9
Glossaire	10
1 Introduction Générale	13
1.1 Contexte et Motivation	13
1.2 Problématique	14
1.3 Objectifs du Projet	14
1.4 Structure du Rapport	14
2 État de l'Art	16
2.1 Introduction	16
2.2 Le Paradigme SDN	16
2.2.1 Historique et Origine	16
2.2.2 Architecture SDN en Couches	17
2.2.3 Avantages du SDN par rapport aux Réseaux Traditionnels . . .	17
2.3 Le Protocole OpenFlow	18
2.3.1 Historique et Principe	18
2.3.2 Structure d'une Table de Flux	18
2.3.3 Champs de Correspondance et Actions	18
2.3.4 Évolution des Versions OpenFlow	19
2.3.5 Fonctionnement du Pipeline OpenFlow	19
2.4 Contrôleurs SDN Open-Source	19

2.4.1	Présentation des Principaux Contrôleurs	19
2.4.2	Tableau Comparatif	20
2.4.3	Justification du Choix de Ryu	20
2.5	Topologies Réseau SDN	21
2.5.1	Topologie en Arbre	21
2.5.2	Topologie Fat-Tree	21
2.6	Sécurité dans les Réseaux SDN	21
2.6.1	Menaces Spécifiques au SDN	21
2.6.2	Attaques DDoS : Types et Mécanismes	22
2.6.3	Méthodes de Détection DDoS	22
2.6.4	Mécanismes de Mitigation	23
2.6.5	Pare-feu Stateful SDN	23
2.6.6	Travaux Existants et Positionnement	23
2.7	Qualité de Service (QoS) dans les SDN	24
2.7.1	Mécanismes QoS Traditionnels	24
2.7.2	QoS Dynamique avec SDN	24
2.7.3	SDN et NFV : Convergence	24
2.8	Vers une Programmation Avancée du Plan de Données	24
2.9	Conclusion	25
3	Architecture Proposée	26
3.1	Vue Générale	26
3.2	Topologie Réseau	27
3.2.1	Topologie Arbre (Référence)	27
3.2.2	Topologie Fat-Tree (Avancée)	27
3.3	Modules Développés	29
3.3.1	Module de Surveillance (monitoring.py)	29
3.3.2	Module de Routage Dynamique (routing.py)	29
3.3.3	Module de Répartition de Charge (load_balancer.py)	29
3.3.4	Module QoS (qos.py)	29
3.3.5	Module de Sécurité (security_manager.py)	30
3.4	Paramètres de Détection DDoS	30
3.5	Conclusion du Chapitre	31
4	Implémentation	32
4.1	Environnement de Développement	32
4.1.1	Outils et Technologies	32
4.1.2	Installation et Configuration	32

4.2	Implémentation du Module de Surveillance	33
4.3	Implémentation du Routage Dynamique	34
4.4	Implémentation du Load Balancer	34
4.5	Implémentation du Module QoS	35
4.6	Implémentation du Module de Sécurité	35
4.6.1	Détection DDoS par Entropie de Shannon	35
4.6.2	Mitigation Automatique par Règle DROP	36
4.6.3	Pare-feu Stateful	36
4.6.4	Règles ACL du Pare-feu	37
4.7	Captures Wireshark des Messages OpenFlow	37
4.8	Conclusion du Chapitre	37
5	Tests et Évaluation des Performances	39
5.1	Environnement de Test	39
5.2	Méthodologie de Comparaison SDN vs Réseau Traditionnel	39
5.3	Scénario 1 : Trafic Normal	40
5.3.1	Description	40
5.3.2	Résultats	40
5.3.3	Analyse	41
5.4	Scénario 2 : Attaque DDoS	41
5.4.1	Description	41
5.4.2	Résultats	41
5.4.3	Analyse	42
5.5	Scénario 3 : Panne d'un Lien	42
5.5.1	Description	42
5.5.2	Résultats	42
5.5.3	Analyse	43
5.6	Scénario 4 : QoS sous Congestion	43
5.6.1	Description	43
5.6.2	Résultats	44
5.6.3	Analyse	44
5.7	Overhead du Contrôleur	44
5.8	Comparaison SDN vs Réseau Traditionnel	45
5.8.1	Analyse Comparative	45
5.9	Comparaison Topologie Arbre vs Fat-Tree k=4	46
5.10	Conclusion du Chapitre	46

6	Conclusion et Perspectives	47
6.1	Bilan du Projet	47
6.2	Résultats Obtenus	47
6.3	Contributions	48
6.4	Difficultés Rencontrées	48
6.5	Perspectives	49
6.5.1	Court Terme	49
6.5.2	Moyen Terme	49
6.5.3	Long Terme	49
6.6	Conclusion Générale	49
A	Guide d’Installation et de Déploiement	53
A.1	Prérequis	53
A.2	Installation Pas à Pas	53
A.3	Lancement des Modules	54
A.4	Tests de Validation	54
B	Structure du Dépôt GitHub	55
C	Paramètres de Configuration	56

Table des figures

3.1	Architecture globale du système SDN proposé	26
3.2	Topologie arbre depth=2, fanout=4 – 5 switches et 16 hôtes	28
3.3	Diagramme des 5 modules et leurs interactions avec le contrôleur Ryu .	28
5.1	Débit TCP selon la charge – SDN vs Réseau Traditionnel	40
5.2	Impact des attaques DDoS sur la latence et le taux de perte	42
5.3	Impact de la panne du lien s1-s2 sur la connectivité	43
5.4	Overhead CPU, mémoire et messages OpenFlow du contrôleur Ryu . .	45

Liste des tableaux

2.1	Architecture en couches SDN et composants utilisés	17
2.2	Comparaison SDN vs réseau traditionnel	17
2.3	Structure d'une entrée de flux OpenFlow	18
2.4	Évolution des versions OpenFlow	19
2.5	Comparaison détaillée des contrôleurs SDN	20
2.6	Types d'attaques DDoS et leurs caractéristiques	22
2.7	Comparaison avec les travaux existants	23
3.1	Architecture en couches du système développé	27
3.2	Politiques QoS implémentées	30
3.3	Paramètres de détection DDoS	31
4.1	Environnement de développement utilisé	32
4.2	États TCP suivis par le pare-feu stateful	37
4.3	Règles ACL du pare-feu stateful	37
5.1	Résultats Scénario 1 – Trafic normal	40
5.2	Métriques UDP et Jitter – Scénario 1	40
5.3	Impact des attaques DDoS sur les performances	41
5.4	Performances du système de détection/mitigation	41
5.5	Impact de la panne du lien s1-s2	42
5.6	Résultats des politiques QoS	44
5.7	Overhead du contrôleur Ryu	44
5.8	Comparaison SDN vs Réseau Traditionnel	45
5.9	Comparaison Topologie Arbre vs Fat-Tree k=4	46
6.1	Récapitulatif des résultats obtenus	47
B.1	Structure du dépôt GitHub	55
C.1	Paramètres configurables des modules	56

Glossaire

ACL (Access Control List) : Liste de règles de filtrage définissant les flux autorisés ou refusés. Dans ce projet, les ACL bloquent Telnet (23), FTP (21), RPC (135), TFTP (69) et autorisent SSH (22), HTTP (80), HTTPS (443), iperf3 (5201) et ICMP.

DDoS (Distributed Denial of Service) : Attaque par déni de service distribué visant à saturer un réseau en envoyant un volume massif de trafic depuis plusieurs sources simultanées. Les types courants comprennent le SYN flood (TCP), l'UDP flood et l'ICMP flood. Dans ce projet, ce type d'attaque est détecté en 5 secondes.

Entropie de Shannon : Mesure statistique de la diversité du trafic réseau définie par la formule $H = -\sum p_i \log_2(p_i)$, normalisée entre 0 et 1. Une entropie proche de 0 indique un trafic concentré (attaque probable), tandis qu'une valeur proche de 1 correspond à un trafic normal et uniformément distribué. Le seuil de détection utilisé dans ce projet est 0.5.

FlowMod : Message OpenFlow (OFPFlowMod) envoyé par le contrôleur aux switches pour installer, modifier ou supprimer des règles de flux dans la table de flux. Ses paramètres clés sont : match, priority, actions, idle_timeout et hard_timeout.

Honeypot (Pot de miel) : Système leurre conçu pour attirer les attaquants afin d'analyser leur comportement. Dans ce projet, h16 (10.0.0.16) joue le rôle de honeypot. Le trafic suspect y est redirigé via l'action SetField d'OpenFlow (priorité 90, timeout 120s).

Mininet : Émulateur de réseau open-source permettant de créer des topologies réseau virtuelles complètes sur une seule machine Linux. Il utilise les namespaces Linux et Open vSwitch pour émuler switches, hôtes et liens. La version 2.3+ est utilisée dans

ce projet.

Mitigation : Ensemble de mesures automatiques prises pour réduire l'impact d'une attaque réseau. Ce projet implémente trois niveaux progressifs : Rate Limiting (priorité 80, 30s), Honeypot (priorité 90, 120s) et DROP total (priorité 100, 60s).

OpenFlow : Protocole de communication standard entre le contrôleur SDN et les switches réseau. Il définit une table de flux composée de règles spécifiant les critères de correspondance (match), la priorité, les actions, les compteurs et les timeouts. La version 1.3, utilisée dans ce projet, introduit les tables multiples et les statistiques améliorées.

OVS (Open vSwitch) : Switch virtuel open-source supportant OpenFlow. Il implémente un datapath kernel Linux hautes performances et est utilisé par Mininet pour émuler les switches OpenFlow dans ce projet.

PacketIn : Message OpenFlow envoyé par un switch au contrôleur lorsqu'un paquet ne correspond à aucune règle de flux (table-miss). Ce message déclenche le traitement par le contrôleur, qui installe ensuite une nouvelle règle via FlowMod.

QoS (Quality of Service) : Ensemble de mécanismes garantissant des niveaux de performance (débit, latence, jitter) pour certains types de flux réseau. Ce projet implémente trois niveaux : HAUTE (iperf3 TCP 5201, SSH), MOYENNE (TCP général, ICMP) et BASSE (UDP).

Ryu : Framework de contrôleur SDN open-source développé en Python par NTT Labs. Il supporte OpenFlow 1.0 à 1.5 et fournit une API événementielle simple basée sur des décorateurs Python. Il est utilisé dans ce projet pour l'implémentation de tous les modules.

SDN (Software-Defined Networking) : Paradigme réseau séparant le plan de contrôle du plan de données. Un contrôleur centralisé dispose d'une vue globale du réseau et programme dynamiquement les switches via OpenFlow. Ce paradigme a été introduit en 2008 à l'Université de Stanford par McKeown et al.

Topologie Fat-Tree : Architecture réseau datacenter organisée en trois niveaux (Core, Agrégation, Edge) offrant une bande passante bisectionnelle maximale et une haute disponibilité grâce aux chemins alternatifs. Pour $k=4$, elle comprend 16 hôtes, 4 pods, 4 switches core, 8 switches agrégation et 8 switches edge.

Chapitre 1

Introduction Générale

1.1 Contexte et Motivation

Les réseaux informatiques constituent aujourd’hui l’épine dorsale de toute infrastructure numérique mondiale. En Afrique de l’Ouest et particulièrement au Sénégal, le développement des infrastructures numériques représente un enjeu stratégique majeur dans le cadre du programme “Sénégal Numérique 2025”. Avec la multiplication des universités connectées et l’essor des services numériques, la gestion efficace et sécurisée des réseaux informatiques est devenue une priorité nationale [12].

Dans ce contexte, les architectures réseau traditionnelles montrent leurs limites face aux exigences croissantes de flexibilité, d’agilité et de sécurité. En effet, ces réseaux présentent plusieurs limitations majeures qu’il convient d’analyser :

- **Rigidité** : La configuration manuelle de chaque équipement rend le réseau difficile à adapter dynamiquement aux variations de trafic, ce qui entraîne des délais importants lors de changements de politique réseau.
- **Complexité de gestion** : L’administration de réseaux hétérogènes nécessite une expertise spécifique pour chaque type d’équipement, sans vue globale cohérente de l’ensemble du réseau.
- **Manque de visibilité globale** : L’absence d’une vue centralisée complique la détection d’anomalies et l’optimisation des performances, car chaque équipement n’a qu’une vision locale du trafic.
- **Sécurité réactive** : Les politiques de sécurité sont configurées statiquement et ne s’adaptent pas en temps réel aux menaces émergentes, laissant le réseau vulnérable pendant les délais de réaction.

Le paradigme SDN (Software-Defined Networking) propose de séparer le plan de contrôle du plan de données via un contrôleur centralisé disposant d’une vision globale du réseau et pouvant programmer dynamiquement les règles de transfert sur les

commutateurs via le protocole OpenFlow [10].

1.2 Problématique

Comment concevoir une architecture SDN capable de gérer dynamiquement le trafic réseau tout en intégrant des mécanismes de détection et de mitigation automatique des attaques réseau ?

Cette problématique soulève plusieurs questions spécifiques auxquelles ce projet apporte des réponses :

1. Comment surveiller en temps réel les flux réseau dans un environnement SDN ?
2. Comment adapter dynamiquement le routage en fonction des métriques de performance ?
3. Comment détecter et atténuer automatiquement les attaques DDoS par analyse statistique du trafic ?
4. Quelles sont les performances d’une architecture SDN par rapport à un réseau traditionnel dans un contexte académique ?

1.3 Objectifs du Projet

- O1** Concevoir une architecture SDN complète identifiant les composants clés et leurs interactions.
- O2** Déployer un environnement de simulation reproductible avec Mininet et le contrôleur Ryu.
- O3** Implémenter un module de gestion dynamique du trafic comprenant le monitoring, le routage adaptatif, la QoS et le load balancing.
- O4** Développer un module de sécurité réseau intégrant la détection DDoS, la mitigation graduée et un pare-feu stateful.
- O5** Évaluer les performances à travers des scénarios de test réalistes et comparer avec un réseau traditionnel simulé dans le même environnement.

1.4 Structure du Rapport

Ce rapport est organisé en six chapitres complémentaires :

- **Chapitre 2** : Présente l’état de l’art sur le SDN, OpenFlow, les contrôleurs et les mécanismes de sécurité réseau.

- **Chapitre 3** : Décrit l'architecture proposée et les quatre modules développés.
- **Chapitre 4** : Détaille l'implémentation de chaque module avec des extraits de code Python.
- **Chapitre 5** : Présente les tests, l'évaluation des performances et la comparaison avec un réseau traditionnel.
- **Chapitre 6** : Conclut le rapport et propose des perspectives d'amélioration.

Chapitre 2

État de l'Art

2.1 Introduction

Ce chapitre présente une revue de la littérature sur le Software-Defined Networking (SDN), les protocoles associés, les contrôleurs existants, les mécanismes de sécurité et la gestion de la qualité de service dans les réseaux programmables. Cette étude bibliographique a guidé les choix architecturaux et technologiques de ce projet.

2.2 Le Paradigme SDN

2.2.1 Historique et Origine

Le concept de Software-Defined Networking a émergé à l'Université de Stanford autour de 2008, avec les travaux fondateurs de McKeown et al. [10] qui ont introduit le protocole OpenFlow. Avant l'avènement du SDN, les réseaux étaient gérés de manière entièrement distribuée : chaque équipement intégrait son propre plan de contrôle, rendant la gestion complexe et peu flexible face aux besoins croissants des infrastructures modernes.

L'idée centrale du SDN consiste à séparer le plan de contrôle du plan de données, ce qui permet une programmation centralisée et flexible du réseau depuis un contrôleur unique disposant d'une vue globale. Selon Kreutz et al. [8], le SDN repose sur quatre piliers fondamentaux :

1. Les plans de données et de contrôle sont découplés et indépendants.
2. Les décisions de transfert des paquets sont basées sur les flux et non sur les destinations.
3. La logique de contrôle est externalisée vers un contrôleur accessible via une API ouverte.

4. Le réseau est entièrement programmable par des applications externes au-dessus du contrôleur.

2.2.2 Architecture SDN en Couches

L'architecture SDN se décompose en trois couches distinctes interconnectées par des interfaces standardisées :

1. **Couche Infrastructure (Data Plane)** : Cette couche regroupe les équipements réseau physiques ou virtuels (switches, routeurs) qui transfèrent les paquets selon les règles reçues du contrôleur. Dans notre projet, elle est émulée par Open vSwitch via Mininet.
2. **Couche Contrôle (Control Plane)** : Le contrôleur SDN constitue le cerveau du réseau. Il maintient une vue globale et cohérente de la topologie, calcule les chemins optimaux et programme dynamiquement les règles de flux via l'interface southbound (OpenFlow). Le contrôleur Ryu joue ce rôle dans notre projet.
3. **Couche Application (Application Plane)** : Les applications réseau (QoS, sécurité, monitoring, load balancing) interagissent avec le contrôleur via l'interface northbound (REST API, Python API). Les modules Python développés dans ce projet constituent cette couche applicative.

TABLE 2.1 – *Architecture en couches SDN et composants utilisés*

Couche	Rôle	Interface	Notre projet
Application	Logique métier	Northbound API	Modules Python
Contrôle	Gestion centralisée	OpenFlow (SB)	Ryu 4.34
Infrastructure	Transfert paquets	OpenFlow (SB)	OVS + Mininet

2.2.3 Avantages du SDN par rapport aux Réseaux Traditionnels

TABLE 2.2 – *Comparaison SDN vs réseau traditionnel*

Critère	Réseau Traditionnel	SDN
Contrôle	Distribué par équipement	Centralisé
Configuration	Manuelle (CLI)	Programmable (API)
Visibilité	Locale	Globale
Adaptabilité	Statique	Dynamique
Sécurité	Réactive	Proactive
Déploiement	Lent	Rapide

2.3 Le Protocole OpenFlow

2.3.1 Historique et Principe

OpenFlow a été proposé par McKeown et al. en 2008 comme premier protocole southbound standard pour le SDN [13]. Il permet au contrôleur de programmer les tables de flux des switches de manière uniforme, indépendamment du constructeur de l'équipement.

2.3.2 Structure d'une Table de Flux

Une table de flux OpenFlow est composée d'entrées structurées selon les champs suivants :

TABLE 2.3 – *Structure d'une entrée de flux OpenFlow*

Champ	Description
Match Fields	Critères d'identification du paquet
Priority	Ordre de traitement (plus élevé = prioritaire)
Counters	Compteurs de paquets et d'octets
Instructions	Actions à appliquer au paquet
Timeouts	idle_timeout et hard_timeout
Cookie	Identifiant de la règle côté contrôleur

2.3.3 Champs de Correspondance et Actions

OpenFlow 1.3 supporte de nombreux champs de correspondance aux couches 2 (MAC, type Ethernet), 3 (IP src/dst, protocole) et 4 (ports TCP/UDP). Les principales actions disponibles sont : Output (envoi vers un port), Drop (suppression du paquet par liste d'actions vide), SetField (modification d'un champ comme la réécriture IP pour le honeypot) et Group (groupe d'actions multipath).

2.3.4 Évolution des Versions OpenFlow

TABLE 2.4 – Évolution des versions OpenFlow

Version	Année	Nouveautés
1.0	2009	Table unique, actions basiques
1.1	2011	Tables multiples, groupes
1.2	2011	Match étendu, IPv6
1.3	2012	Meters, tunnels, améliorations stats
1.4	2013	Synchronisation, bundles
1.5	2014	Egress tables, statistiques avancées

Ce projet utilise **OpenFlow 1.3** pour sa stabilité, son support des statistiques avancées (port_stats, flow_stats) et sa large adoption dans la communauté académique [13].

2.3.5 Fonctionnement du Pipeline OpenFlow

Lorsqu'un paquet arrive sur un switch OpenFlow, celui-ci consulte sa table de flux. Si une règle correspond au paquet (match), les actions associées sont immédiatement appliquées. Si aucune règle ne correspond (table-miss), le switch envoie le paquet au contrôleur via un message PacketIn. Le contrôleur analyse alors le paquet, prend une décision de routage ou de sécurité, et installe une nouvelle règle de flux via FlowMod. Les paquets suivants appartenant au même flux sont ensuite traités directement par le switch sans solliciter le contrôleur.

2.4 Contrôleurs SDN Open-Source

2.4.1 Présentation des Principaux Contrôleurs

Ryu

Ryu est un framework de contrôleur SDN open-source développé en Python par NTT Labs [14]. Il supporte OpenFlow 1.0 à 1.5 et fournit une API événementielle simple basée sur des décorateurs Python, ce qui facilite le développement rapide d'applications réseau personnalisées.

ONOS

ONOS (Open Network Operating System) est un contrôleur SDN haute performance développé en Java par l'Open Networking Foundation (ONF). Il est conçu pour

les déploiements en production avec haute disponibilité, scalabilité et support multi-contrôleurs.

OpenDaylight

OpenDaylight est un contrôleur SDN open-source soutenu par la Linux Foundation. Son architecture modulaire via OSGi/Karaf supporte de nombreux protocoles southbound (OpenFlow, NETCONF, OVSDB), mais sa complexité de configuration en limite l’usage académique.

Floodlight

Floodlight est un contrôleur SDN Java développé par Big Switch Networks. Il propose une API REST simple et un bon support d’OpenFlow 1.0 à 1.3, mais son développement a ralenti ces dernières années.

2.4.2 Tableau Comparatif

TABLE 2.5 – *Comparaison détaillée des contrôleurs SDN*

Critère	Ryu	ONOS	ODL	Floodlight
Langage	Python	Java	Java	Java
OpenFlow	1.0-1.5	1.0-1.5	1.0-1.5	1.0-1.3
Scalabilité	Moyenne	Haute	Haute	Moyenne
Complexité	Faible	Haute	Très haute	Moyenne
Communauté	Académique	Industrielle	Industrielle	Mixte
Intégration Mininet	Excellente	Bonne	Bonne	Bonne

2.4.3 Justification du Choix de Ryu

Ryu a été retenu pour ce projet pour plusieurs raisons complémentaires. Sa simplicité d’implémentation, grâce à une API Python basée sur des décorateurs, permet un développement rapide et lisible. Son support complet d’OpenFlow 1.0 à 1.5, son intégration native avec Mininet et sa large adoption dans la communauté académique en font un choix idéal pour le prototypage. Enfin, son faible overhead (21,5% CPU, 65 MB mémoire) le rend adapté aux environnements contraints [4].

2.5 Topologies Réseau SDN

2.5.1 Topologie en Arbre

La topologie en arbre est la plus utilisée dans les environnements SDN académiques. Elle se caractérise par un switch racine connecté à des switches intermédiaires, auxquels sont reliés les hôtes finaux. Si elle présente l'avantage d'une structure simple et facile à administrer, elle souffre d'un inconvénient majeur : il n'existe qu'un seul chemin entre toute paire source-destination, ce qui la rend vulnérable aux pannes de liens.

Dans ce projet, la topologie arbre $\text{depth}=2$, $\text{fanout}=4$ comprend 5 switches et 16 hôtes, offrant un bon compromis entre complexité et représentativité.

2.5.2 Topologie Fat-Tree

La topologie Fat-Tree, proposée par Al-Fares et al. [3], constitue une architecture datacenter à trois niveaux (Core, Agrégation, Edge) qui offre une bande passante bisectionnelle maximale, des chemins multiples $(k/2)^2$ entre tout pod source et destination, une tolérance aux pannes naturelle grâce à la redondance, et une scalabilité linéaire en $k^3/4$ hôtes.

2.6 Sécurité dans les Réseaux SDN

2.6.1 Menaces Spécifiques au SDN

Le paradigme SDN introduit de nouvelles vulnérabilités spécifiques [15]. La centralisation du contrôleur crée un point unique de défaillance qui constitue une cible privilégiée des attaquants. La saturation du canal de contrôle par un flood de messages PacketIn peut paralyser le contrôleur. La manipulation des règles de flux par injection de règles malveillantes et les attaques DDoS saturant le réseau représentent les principales menaces identifiées dans les SDN.

2.6.2 Attaques DDoS : Types et Mécanismes

TABLE 2.6 – *Types d'attaques DDoS et leurs caractéristiques*

Type	Protocole	Mécanisme
SYN flood	TCP	Connexions semi-ouvertes
UDP flood	UDP	Saturation bande passante
ICMP flood	ICMP	Ping massif
HTTP flood	TCP/HTTP	Requêtes web massives
Amplification	UDP/DNS	Réflexion et amplification

2.6.3 Méthodes de Détection DDoS

Détection par Entropie de Shannon

L'entropie de Shannon mesure la diversité d'une distribution de probabilités. Appliquée aux adresses IP du trafic réseau, une baisse soudaine de l'entropie indique une concentration anormale caractéristique d'une attaque DDoS [7]. La formule de l'entropie normalisée est :

$$H_{norm} = \frac{-\sum_{i=1}^n p_i \log_2(p_i)}{\log_2(n)} \quad (2.1)$$

où p_i désigne la proportion de trafic associée à l'adresse IP i , et n représente le nombre total d'adresses IP distinctes observées dans l'intervalle d'analyse.

Détection par Machine Learning

Les travaux récents démontrent que les algorithmes d'apprentissage automatique améliorent considérablement la précision de détection des attaques DDoS dans les SDN. Tang et al. [17] obtiennent des taux de détection supérieurs à 99% avec l'algorithme Random Forest appliqué aux statistiques de flux OpenFlow. Xiao et al. [18] montrent l'efficacité des réseaux de neurones profonds (LSTM, CNN) pour la détection en temps réel, tandis qu'Ahuja et al. [2] proposent une revue complète des techniques de détection DDoS dans les SDN, soulignant les avantages comparatifs des approches hybrides entropie/ML.

Détection par Analyse des Flux

Le nombre de destinations uniques contactées par une même IP source constitue un indicateur efficace pour détecter les scans de ports et les attaques distribuées [11].

Un seuil de 10 destinations uniques par IP source dans un intervalle de 5 secondes déclenche une alerte de type scan dans notre implémentation.

2.6.4 Mécanismes de Mitigation

Les travaux de Shin et al. [16] sur AVANT-GUARD montrent qu’une mitigation efficace dans les SDN doit combiner la rapidité de réaction et la précision du ciblage pour éviter les blocages injustifiés. Boite et al. [5] proposent une approche graduée basée sur l’analyse d’anomalies de flux, proche de notre implémentation à trois niveaux progressifs : le blocage par règle DROP, la redirection vers un honeypot pour capturer le comportement de l’attaquant, et le rate limiting par règles à timeout court.

2.6.5 Pare-feu Stateful SDN

Contrairement aux pare-feux traditionnels qui opèrent de manière distribuée sur chaque équipement, un pare-feu stateful SDN centralise le suivi des connexions TCP au niveau du contrôleur et installe dynamiquement des règles de filtrage adaptées sur les switches. Les états TCP gérés dans ce projet sont : SYN (nouvelle connexion initiée), ESTABLISHED (connexion confirmée par SYN-ACK) et CLOSING (connexion en cours de fermeture par FIN).

2.6.6 Travaux Existants et Positionnement

TABLE 2.7 – *Comparaison avec les travaux existants*

Travail	Méthode	Mitigation	QoS
Braga et al. (2010) [7]	Clustering	DROP	Non
Shin et al. (2013) [16]	Statistiques	DROP	Non
Mousavi et al. (2015) [11]	Entropie	DROP	Non
Tang et al. (2020) [17]	ML/Random Forest	DROP	Non
Boite et al. (2021) [5]	Anomalie flux	Graduée	Non
Ce projet	Entropie + seuil	DROP + Honeypot + RL	Oui

Notre approche se distingue par la combinaison de la détection par entropie avec une mitigation graduée en trois niveaux et l’intégration simultanée d’un module QoS, une combinaison qui n’est proposée par aucun des travaux existants recensés.

2.7 Qualité de Service (QoS) dans les SDN

2.7.1 Mécanismes QoS Traditionnels

Dans les réseaux traditionnels, la QoS est implémentée via plusieurs mécanismes complémentaires. DiffServ (Differentiated Services) repose sur le marquage DSCP des paquets pour les classer en différentes classes de service. IntServ (Integrated Services) réserve des ressources bout-en-bout via le protocole RSVP (Resource Reservation Protocol). Les files d'attente prioritaires telles que WFQ (Weighted Fair Queuing), PQ (Priority Queuing) et CBWFQ (Class-Based WFQ) permettent de différencier le traitement des flux au niveau de chaque interface réseau. Ces mécanismes sont configurés statiquement sur chaque équipement, sans adaptabilité dynamique aux changements de conditions réseau.

2.7.2 QoS Dynamique avec SDN

Le SDN apporte une révolution dans la gestion de la QoS en permettant une approche centralisée et dynamique [9]. Le contrôleur classe les flux et installe les règles de priorité sur tous les switches simultanément, sans intervention manuelle. Les politiques QoS peuvent être modifiées en temps réel en réponse aux variations de trafic ou aux priorités applicatives. La priorisation s'effectue au niveau du flux individuel (5-tuple : IP src/dst, port src/dst, protocole), offrant une granularité bien supérieure aux mécanismes DiffServ. Dans ce projet, trois niveaux de QoS sont définis et appliqués via des règles OpenFlow de priorités 10, 15, 20, 25 et 30.

2.7.3 SDN et NFV : Convergence

Le SDN s'intègre naturellement avec la virtualisation des fonctions réseau (NFV – Network Functions Virtualization). Cette convergence SDN/NFV permet de déployer des fonctions réseau comme les pare-feux, les load balancers et les systèmes de détection d'intrusion sous forme de machines virtuelles ou de conteneurs, orchestrées dynamiquement par le contrôleur SDN selon les besoins du trafic.

2.8 Vers une Programmation Avancée du Plan de Données

Le protocole P4 (Programming Protocol-Independent Packet Processors) [6] représente une évolution majeure du paradigme SDN en permettant de programmer directe-

ment le comportement du plan de données des switches, indépendamment du protocole utilisé. Contrairement à OpenFlow dont le comportement des switches est fixé par la spécification du protocole, P4 permet de définir entièrement les champs analysés, les tables de flux et les actions appliquées. Cette flexibilité ouvre la voie à des implémentations de détection DDoS directement dans le datapath réseau, avec des temps de réaction de l'ordre de la nanoseconde, sans solliciter le contrôleur.

2.9 Conclusion

Cet état de l'art confirme que le SDN offre des avantages significatifs sur les réseaux traditionnels en termes de flexibilité, programmabilité et gestion proactive de la sécurité. Le choix du contrôleur Ryu, du protocole OpenFlow 1.3 et de l'émulateur Mininet est pleinement justifié par leur maturité, leur complémentarité et leur large adoption dans la communauté académique [4]. La littérature récente sur la détection DDoS par machine learning (2020-2025) ouvre des perspectives d'amélioration directes pour ce projet. Notre contribution originale réside dans la combinaison inédite de la détection par entropie avec une mitigation graduée en trois niveaux et un module QoS intégré.

Chapitre 3

Architecture Proposée

3.1 Vue Générale

L'architecture SDN proposée s'organise en trois couches conformément au modèle SDN standard, avec quatre modules applicatifs déployés sur le contrôleur Ryu. Chaque module est implémenté sous forme d'une application Ryu autonome pouvant être lancée indépendamment ou en combinaison avec les autres :

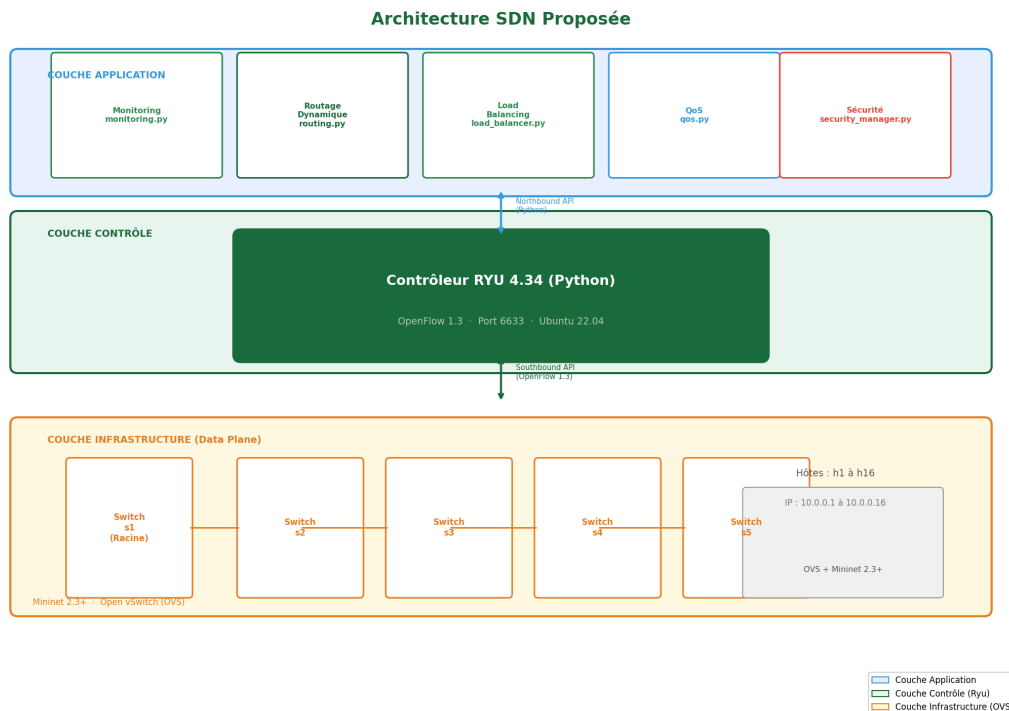


FIGURE 3.1 – Architecture globale du système SDN proposé

TABLE 3.1 – *Architecture en couches du système développé*

Couche	Composant	Rôle
Application	Module QoS, Module Sécurité	Logique métier
Contrôle	Contrôleur Ryu (Python)	Gestion centralisée
Infrastructure	Switches OVS (Mininet)	Transfert paquets

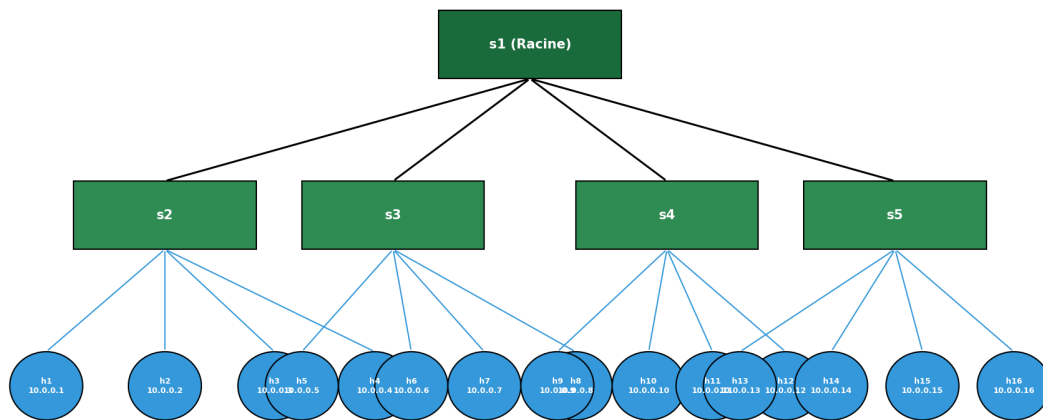
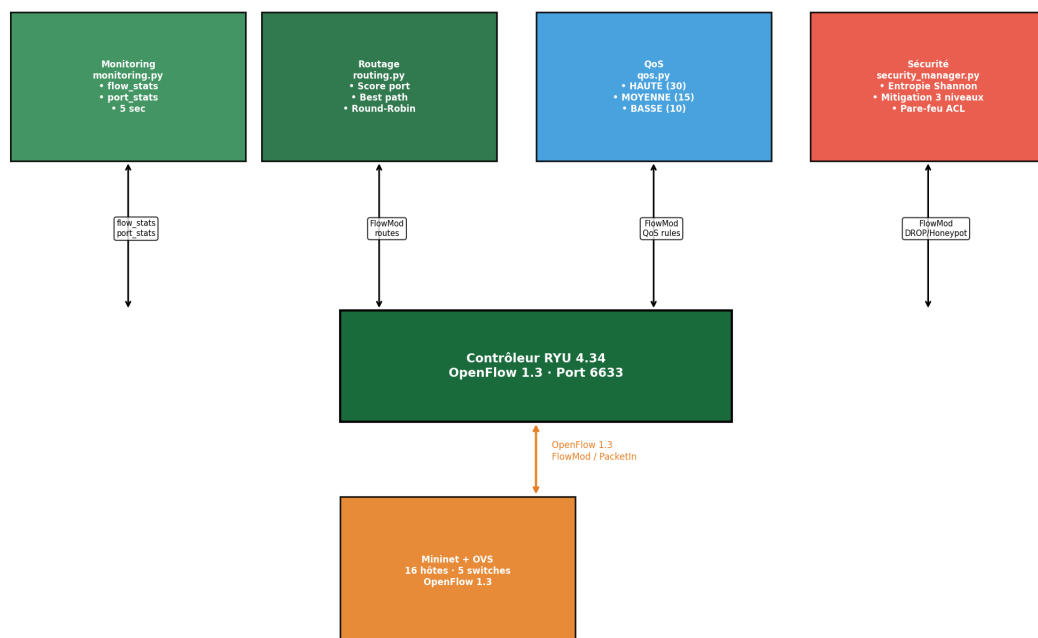
3.2 Topologie Réseau

3.2.1 Topologie Arbre (Référence)

La topologie principale utilisée pour les tests est une topologie arbre (depth=2, fanout=4). Cette topologie comprend 1 switch racine (s1), 4 switches intermédiaires (s2, s3, s4, s5) et 16 hôtes (h1 à h16) dont les adresses IP s'étendent de 10.0.0.1 à 10.0.0.16. Elle offre un bon compromis entre complexité de gestion et représentativité pour les tests de performance et de sécurité.

3.2.2 Topologie Fat-Tree (Avancée)

Une topologie Fat-Tree k=4 a également été implémentée dans `fat_tree.py`. Contrairement à la topologie arbre, le Fat-Tree offre des chemins multiples entre toute paire source-destination, ce qui permet un reroutage automatique en cas de panne de lien et une meilleure utilisation de la bande passante disponible.

Topologie Arbre (depth=2, fanout=4) - 5 switches, 16 hôtes**FIGURE 3.2** – Topologie arbre depth=2, fanout=4 – 5 switches et 16 hôtes**Modules SDN et Interactions avec le Contrôleur Ryu****FIGURE 3.3** – Diagramme des 5 modules et leurs interactions avec le contrôleur Ryu

3.3 Modules Développés

3.3.1 Module de Surveillance (monitoring.py)

Ce module assure la collecte périodique des statistiques OpenFlow de tous les switches connectés toutes les 5 secondes. Il recueille les statistiques par règle de flux (flow_stats : paquets, octets, priorité) et les statistiques par port physique (port_stats : rx/tx paquets et octets). Ces métriques alimentent en temps réel les algorithmes de routage dynamique et de détection DDoS.

3.3.2 Module de Routage Dynamique (routing.py)

Ce module implémente un routage adaptatif en sélectionnant le port de sortie optimal selon un score de qualité calculé pour chaque port disponible :

$$score = \text{taux_perte}(\%) \times 2 + \text{débit}(MB/s) \quad (3.1)$$

Le port affichant le score le plus bas est sélectionné, car il combine un faible taux de perte et une charge modérée. Le taux de perte est pondéré deux fois plus que le débit afin de pénaliser davantage les liens instables que les liens simplement chargés.

3.3.3 Module de Répartition de Charge (load_balancer.py)

Ce module distribue les flux réseau équitablement entre les ports disponibles via l'algorithme Round-Robin. Chaque nouveau flux (identifié par le couple src_mac, dst_mac) se voit assigner un port de sortie de manière cyclique :

$$port = ports[\text{compteur} \bmod nb_ports] \quad (3.2)$$

L'assignation est mémorisée pour garantir que tous les paquets d'un même flux empruntent le même chemin, assurant ainsi la cohérence et l'ordre de livraison des paquets.

3.3.4 Module QoS (qos.py)

Ce module installe des règles de priorisation sur les switches dès leur connexion au contrôleur, garantissant une différenciation de service dès le premier paquet. Trois niveaux de priorité sont définis :

TABLE 3.2 – *Politiques QoS implémentées*

Niveau	Priorité OF	Trafic	Port
HAUTE	30	iperf3 TCP	5201
HAUTE	25	SSH	22
MOYENNE	20	TCP général	*
MOYENNE	15	ICMP	*
BASSE	10	UDP	*

3.3.5 Module de Sécurité (security_manager.py)

Ce module unifié intègre trois sous-systèmes de sécurité complémentaires qui opèrent de manière coordonnée.

Détection DDoS

La détection repose sur trois algorithmes complémentaires appliqués toutes les 5 secondes : le calcul de l'entropie de Shannon des distributions d'adresses IP source et destination, la mesure du taux de paquets par seconde par IP source, et le comptage des destinations uniques par IP source pour identifier les scans de ports.

Mitigation Automatique Graduée

La mitigation est appliquée progressivement selon la sévérité de la menace détectée, en trois niveaux : Rate Limiting (priorité 80, 30s) pour limiter le débit, Honeypot (priorité 90, 120s) pour rediriger le trafic vers h16 (10.0.0.16) via SetField, et DROP (priorité 100, 60s) pour bloquer totalement l'IP attaquante.

Pare-feu Stateful

Le pare-feu suit les états des connexions TCP (SYN, ESTABLISHED, CLOSING) et applique des règles ACL configurables bloquant les protocoles non sécurisés (Telnet, FTP, RPC, TFTP) tout en autorisant les services légitimes (SSH, HTTP, HTTPS, iperf3).

3.4 Paramètres de Détection DDoS

Le seuil de **2 paquets par seconde** a été délibérément choisi en adéquation avec l'environnement Mininet simulé. Dans cet environnement, le trafic de fond généré spontanément par les 16 hôtes de la topologie reste inférieur à 1 pkt/s par IP source, ce

qui rend le seuil de 2 pkt/s pertinent pour détecter les floods dès quelques dizaines de paquets par seconde, tout en évitant les faux positifs. Comme le recommandent Braga et al. [7], ce seuil devrait être recalibré empiriquement sur au moins 24 heures de trafic normal avant tout déploiement en environnement réel.

TABLE 3.3 – Paramètres de détection DDoS

Paramètre	Valeur	Description
SEUIL_PAQUETS_PAR_SEC	2	Alerte si > 2 pkt/s
SEUIL_ENTROPIE	0.5	Alerte si entropie < 0.5
SEUIL_FLUX_PAR_IP	10	Alerte si > 10 destinations
INTERVALLE_ANALYSE	5s	Période d'analyse

3.5 Conclusion du Chapitre

Ce chapitre a présenté l'architecture SDN proposée, articulée autour de quatre modules complémentaires déployés sur le contrôleur Ryu : monitoring, routage dynamique, QoS et sécurité. Cette architecture modulaire offre une gestion centralisée et programmable du réseau, avec des mécanismes de sécurité proactifs adaptés aux menaces contemporaines. Les algorithmes de score de port et d'entropie de Shannon constituent les éléments techniques centraux de cette architecture. Le chapitre suivant détaille l'implémentation de chacun de ces modules en Python avec Ryu et Mininet.

Chapitre 4

Implémentation

4.1 Environnement de Développement

4.1.1 Outils et Technologies

TABLE 4.1 – *Environnement de développement utilisé*

Composant	Version/Détail
Système d'exploitation	Ubuntu 22.04 LTS
Émulateur réseau	Mininet 2.3+
Contrôleur SDN	Ryu 4.34
Langage	Python 3.12
Protocole	OpenFlow 1.3
Switch virtuel	Open vSwitch (OVS)
Génération trafic	iperf3, hping3
Monitoring	tshark, Wireshark
Versioning	Git / GitHub

4.1.2 Installation et Configuration

L'environnement SDN a été déployé dans un environnement virtuel Python afin d'isoler les dépendances et de garantir la reproductibilité des expériences sur différentes machines :

```
1 # Creation de l'environnement virtuel Python
2 python3 -m venv ~/projet-sdn/venv
3 source ~/projet-sdn/venv/bin/activate
4
5 # Installation du controleur Ryu et dependances
6 pip install ryu matplotlib numpy
```

```

7
8 # Lancement du controleur SDN
9 ryu-manager traffic_manager.py
10
11 # Lancement de Mininet avec topologie arbre
12 sudo mn --controller=remote,ip=127.0.0.1,port=6633 \
13         --switch=ovsk,protocols=OpenFlow13 \
14         --topo=tree,depth=2,fanout=4 --mac
15
16 # Verification de la connectivite initiale
17 mininet> pingall

```

Listing 4.1 – Installation de l’environnement SDN

4.2 Implémentation du Module de Surveillance

Le module de monitoring collecte périodiquement les statistiques OpenFlow via des requêtes OFPPortStatsRequest toutes les 5 secondes. Pour chaque port de chaque switch, il calcule le débit instantané en bytes par seconde par différence entre deux relevés successifs de compteurs, et le taux de perte de paquets comme rapport entre les paquets perdus et les paquets émis :

```

1 def _port_stats_reply_handler(self, ev):
2     dpid = ev.msg.datapath.id
3     for stat in ev.msg.body:
4         port = stat.port_no
5         # Calcul du debit instantane (bytes/sec)
6         curr = stat.tx_bytes + stat.rx_bytes
7         speed = (curr - self.prev_bytes[dpid][port]) / 5.0
8         self.prev_bytes[dpid][port] = curr
9         # Calcul du taux de perte de paquets
10        loss = 0.0
11        if stat.tx_packets > 0:
12            loss = max(0.0,
13                      (stat.tx_packets - stat.rx_packets)
14                      / stat.tx_packets * 100)
15        self.port_stats[dpid][port] = {
16            'speed': speed, 'loss': loss}

```

Listing 4.2 – Calcul du débit et taux de perte (traffic_manager.py)

4.3 Implémentation du Routage Dynamique

Le module de routage sélectionne le port de sortie optimal en calculant un score pour chaque port disponible. Ce score combine le taux de perte (pondéré $\times 2$) et le débit normalisé en MB/s, favorisant les liens à la fois fiables et peu chargés :

```

1 def _best_port(self, dpid, available_ports):
2     best_port = None
3     best_score = float('inf')
4     for port in available_ports:
5         metrics = self.port_stats[dpid].get(port, {})
6         speed = metrics.get('speed', 0)
7         loss = metrics.get('loss', 0)
8         # Score : perte ponderee x2 + charge normalisee
9         score = loss * 2 + (speed / 1e6)
10        if score < best_score:
11            best_score = score
12            best_port = port
13    return best_port

```

Listing 4.3 – Sélection du meilleur port (*routing.py*)

4.4 Implémentation du Load Balancer

L'algorithme Round-Robin assigne cycliquement un port de sortie à chaque nouveau flux, en mémorisant l'assignation pour garantir la cohérence des paquets :

```

1 def _assign_port_rr(self, dpid, src, dst, available_ports):
2     key = (src, dst)
3     if key not in self.flow_port[dpid]:
4         # Selection cyclique du port
5         idx = self.rr_counter[dpid] % len(available_ports)
6         self.rr_counter[dpid] += 1
7         self.flow_port[dpid][key] = available_ports[idx]
8         self.port_load[dpid][available_ports[idx]] += 1
9     return self.flow_port[dpid][key]

```

Listing 4.4 – Algorithme Round-Robin (*load_balancer.py*)

4.5 Implémentation du Module QoS

Les règles QoS sont installées sur les switches via FlowMod dès leur connexion au contrôleur, garantissant une priorisation du trafic dès le premier paquet reçu :

```

1 def _install_qos_rules(self, datapath):
2     parser = datapath.ofproto_parser
3     ofproto = datapath.ofproto
4     actions = [parser.OFPACTIONOutput(ofproto.OFPP_NORMAL)]
5
6     # HAUTE priorite (30) : iperf3 TCP port 5201
7     match = parser.OFPMATCH(eth_type=0x0800,
8                             ip_proto=6, tcp_dst=5201)
9     self.add_flow(datapath, 30, match, actions)
10
11    # HAUTE priorite (25) : SSH TCP port 22
12    match = parser.OFPMATCH(eth_type=0x0800,
13                            ip_proto=6, tcp_dst=22)
14    self.add_flow(datapath, 25, match, actions)
15
16    # MOYENNE priorite (15) : ICMP
17    match = parser.OFPMATCH(eth_type=0x0800, ip_proto=1)
18    self.add_flow(datapath, 15, match, actions)
19
20    # BASSE priorite (10) : UDP
21    match = parser.OFPMATCH(eth_type=0x0800, ip_proto=17)
22    self.add_flow(datapath, 10, match, actions)

```

Listing 4.5 – Installation des règles QoS (*qos.py*)

4.6 Implémentation du Module de Sécurité

4.6.1 Détection DDoS par Entropie de Shannon

L'algorithme calcule l'entropie de Shannon normalisée des distributions d'adresses IP source et destination pour chaque intervalle d'analyse de 5 secondes. Une valeur proche de 0 déclenche une alerte DDoS :

```

1 def _calcul_entropie(self, compteurs):
2     total = sum(compteurs.values())
3     if total == 0:

```

```

4         return 1.0 # Pas de trafic = entropie maximale
5     entropie = 0.0
6     for count in compteurs.values():
7         p = count / total
8         if p > 0:
9             entropie -= p * math.log2(p)
10    # Normalisation par log2(N) pour obtenir [0,1]
11    nb = len(compteurs)
12    if nb > 1:
13        entropie /= math.log2(nb)
14    return entropie

```

Listing 4.6 – Calcul de l'entropie de Shannon (*ddos_detector.py*)

4.6.2 Mitigation Automatique par Règle DROP

Lorsqu'une IP source est identifiée comme attaquante, une règle OpenFlow de haute priorité est installée avec une liste d'actions vide (DROP) et un `hard_timeout` de 60 secondes, assurant un blocage temporaire :

```

1 def _block_ip(self, datapath, ip_src):
2     # Verification : IP deja bloquee sur ce switch ?
3     if ip_src in self.blocked_ips[datapath.id]:
4         return
5     parser = datapath.ofproto_parser
6     # Correspondance sur l'adresse IP source IPv4
7     match = parser.OFPMatch(eth_type=0x0800,
8                             ipv4_src=ip_src)
9     # Action DROP : liste vide, priorite 100, timeout 60s
10    self.add_flow(datapath, 100, match, [], hard=60)
11    self.blocked_ips[datapath.id].add(ip_src)

```

Listing 4.7 – Blocage DROP d'une IP suspecte (*ddos_detector.py*)

4.6.3 Pare-feu Stateful

Le pare-feu suit dynamiquement les états des connexions TCP :

TABLE 4.2 – États TCP suivis par le pare-feu stateful

Drapeau TCP	État	Action
SYN	Nouvelle connexion	Vérification ACL
SYN-ACK	Connexion établie	ESTABLISHED
FIN	Fermeture	CLOSING
Données	Connexion active	Mise à jour timestamp

4.6.4 Règles ACL du Pare-feu

TABLE 4.3 – Règles ACL du pare-feu stateful

Action	Protocole	Port	Service
DENY	TCP	23	Telnet
DENY	TCP	21	FTP
DENY	TCP	135	RPC
DENY	UDP	69	TFTP
ALLOW	TCP	22	SSH
ALLOW	TCP	80	HTTP
ALLOW	TCP	443	HTTPS
ALLOW	TCP	5201	iperf3
ALLOW	ICMP	*	Ping
ALLOW	*	*	Défaut

4.7 Captures Wireshark des Messages OpenFlow

La surveillance des échanges OpenFlow entre le contrôleur Ryu et les switches a été réalisée avec Wireshark sur l'interface loopback avec le filtre `openflow_v4`. Les messages observés comprennent les requêtes de statistiques `OFPT_MULTIPART_REQUEST` envoyées par Ryu toutes les 5 secondes, les réponses `OFPT_MULTIPART_REPLY` des switches avec les compteurs de ports et flux, les messages keepalive `OFPT_ECHO_REPLY` pour maintenir la connexion, et les messages `OFPT_FLOW_MOD` pour l'installation des règles de flux. Au total, 30 messages OpenFlow ont été échangés en 10 secondes, confirmant un overhead de contrôle minimal de 3 messages par seconde.

4.8 Conclusion du Chapitre

Ce chapitre a présenté l'implémentation complète des quatre modules de l'architecture SDN en Python avec Ryu. Les extraits de code illustrent les algorithmes techniques clés : le calcul de l'entropie de Shannon pour la détection DDoS, la sélection du meilleur

port par score pour le routage adaptatif, l'algorithme Round-Robin pour le load balancing, et l'installation des règles QoS et de sécurité via FlowMod OpenFlow. La section suivante évalue les performances de cette implémentation sur quatre scénarios de test distincts.

Chapitre 5

Tests et Évaluation des Performances

5.1 Environnement de Test

Les tests ont été réalisés sur la topologie arbre (depth=2, fanout=4) avec 16 hôtes et 5 switches OVS connectés au contrôleur Ryu via OpenFlow 1.3. Les outils de mesure utilisés sont ping (latence et taux de perte), iperf3 (débit TCP/UDP et jitter), hping3 (génération d'attaques DDoS) et tshark/Wireshark (capture et analyse des messages OpenFlow).

5.2 Méthodologie de Comparaison SDN vs Réseau Traditionnel

Afin de garantir une comparaison scientifiquement valide, le réseau traditionnel a été simulé dans **le même environnement Mininet** avec la même topologie arbre (depth=2, fanout=4) et les mêmes paramètres de bande passante. La seule différence introduite est l'absence de contrôleur SDN, obtenue par l'option `-controller=none`. Dans cette configuration, les switches OVS fonctionnent en mode autonome avec un simple apprentissage MAC basique, sans optimisation de routage, sans politiques QoS et sans aucun mécanisme de sécurité. Les mêmes outils de mesure (iperf3, ping, hping3) ont été utilisés dans les deux configurations avec des paramètres identiques, afin de garantir la comparabilité des résultats obtenus.

5.3 Scénario 1 : Trafic Normal

5.3.1 Description

Ce scénario évalue les performances du réseau SDN en conditions normales, avec trois niveaux de charge croissants : faible (1 flux TCP), moyenne (4 flux simultanés) et forte (8 flux simultanés).

5.3.2 Résultats

TABLE 5.1 – *Résultats Scénario 1 – Trafic normal*

Charge	Flux	Débit TCP	Latence avg	Perte
Faible	1 flux	23,5 Gbps	0,134 ms	0%
Moyenne	4 flux	24,2 Gbps	–	0%
Forte	8 flux	21,5 Gbps	–	0%

TABLE 5.2 – *Métriques UDP et Jitter – Scénario 1*

Débit UDP	Jitter	Perte UDP	Paquets
987 Mbits/sec	0,001 ms	1,2%	863 247

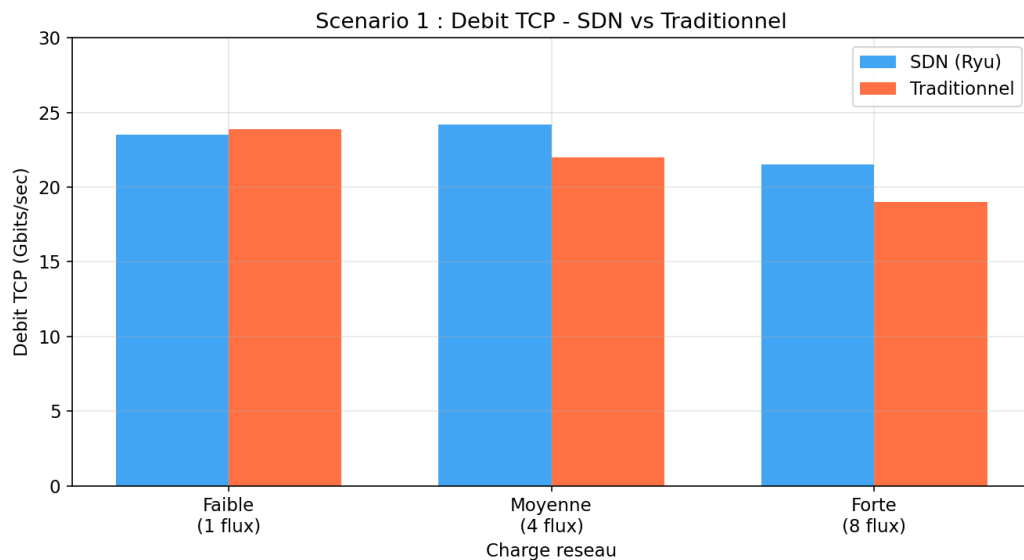


FIGURE 5.1 – *Débit TCP selon la charge – SDN vs Réseau Traditionnel*

5.3.3 Analyse

Les résultats démontrent d'excellentes performances du réseau SDN en conditions normales. Le débit TCP se maintient entre 21,5 et 24,2 Gbps quelle que soit la charge appliquée, ce qui confirme la capacité du routage dynamique à préserver des performances élevées sous charge variable. Le jitter UDP de 0,001 ms témoigne d'une très faible variabilité de la latence, une qualité essentielle pour les applications temps réel et la téléphonie sur IP.

5.4 Scénario 2 : Attaque DDoS

5.4.1 Description

Ce scénario évalue l'impact des attaques DDoS sur les performances réseau et l'efficacité du système de détection et mitigation automatique. Deux types d'attaques ont été générés depuis h1 : un ICMP flood par envoi rapide de pings (`ping -i 0.01`) et un SYN+UDP flood combiné via hping3 (`hping3 -S -p 80 -flood` et `hping3 -2 -p 53 -flood`).

5.4.2 Résultats

TABLE 5.3 – *Impact des attaques DDoS sur les performances*

Phase	Latence avg	Latence max	Perte
Avant attaque	0,478 ms	1,966 ms	0%
ICMP flood	4,793 ms	45,378 ms	0%
SYN+UDP flood	12 718 ms	16 833 ms	10%
Après mitigation	2,389 ms	28,498 ms	0%

TABLE 5.4 – *Performances du système de détection/mitigation*

Métrique	Valeur
Temps de détection	5 secondes
Temps de mitigation	< 1 seconde
Débit attaque détecté	446 pkt/s
Seuil de déclenchement	2 pkt/s
Durée du blocage DROP	60 secondes

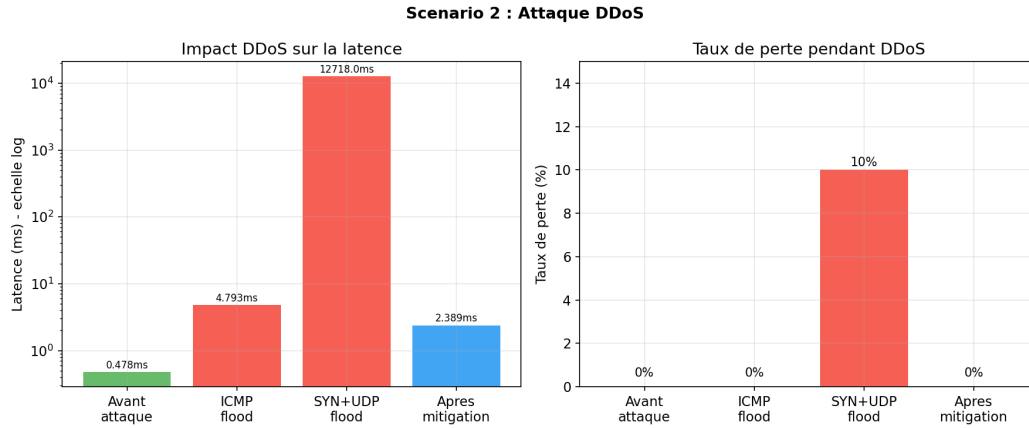


FIGURE 5.2 – Impact des attaques DDoS sur la latence et le taux de perte

5.4.3 Analyse

Le système de détection identifie les attaques en un seul intervalle d'analyse de 5 secondes et déclenche la mitigation automatique en moins d'une seconde après détection. Après l'installation de la règle DROP sur l'IP attaquante, la latence revient à 2,389 ms, contre 0,478 ms avant l'attaque. Cette légère différence résiduelle s'explique par la congestion accumulée pendant les 5 secondes précédant la détection.

5.5 Scénario 3 : Panne d'un Lien

5.5.1 Description

Ce scénario simule la panne physique du lien entre les switches s1 et s2 par la commande Mininet link s1 s2 down, puis son rétablissement par link s1 s2 up. Ce test évalue la résilience du réseau SDN face aux défaillances d'infrastructure.

5.5.2 Résultats

TABLE 5.5 – Impact de la panne du lien s1-s2

Phase	Latence avg	Perte
Avant panne	5,056 ms	0%
Pendant panne	—	100%
Après rétablissement	6,086 ms	0%

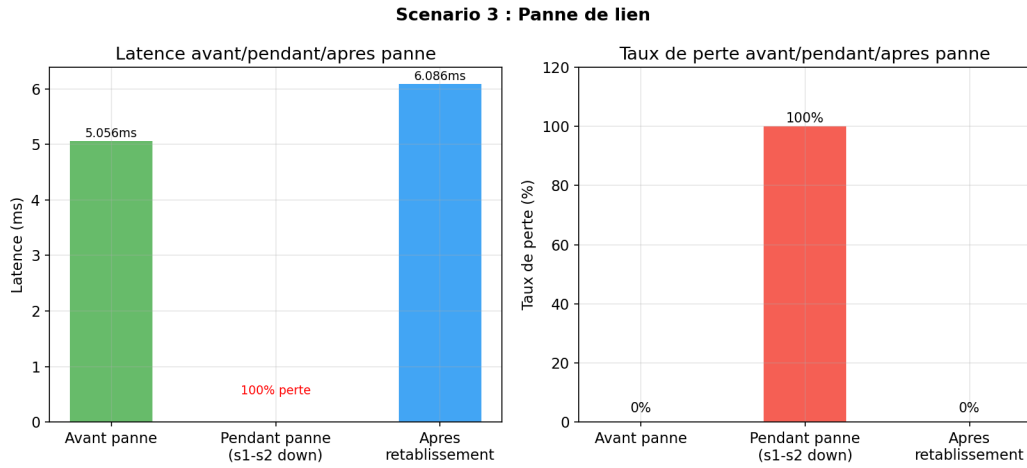


FIGURE 5.3 – Impact de la panne du lien s1-s2 sur la connectivité

5.5.3 Analyse

La topologie arbre ne disposant que d'un seul chemin entre h1 et h16 via le lien s1-s2, la panne entraîne une interruption totale de la connectivité (100% de perte). Après rétablissement du lien, la connectivité est restaurée avec une latence légèrement augmentée (6,086 ms vs 5,056 ms avant panne), due au délai nécessaire à la réinstallation des règles de flux par le contrôleur. La topologie Fat-Tree k=4 permettrait quant à elle un reroutage automatique vers les chemins alternatifs disponibles, sans interruption de service.

5.6 Scénario 4 : QoS sous Congestion

5.6.1 Description

Ce scénario évalue l'application des politiques QoS en présence d'un trafic mixte simultané : un flux TCP iperf3 en haute priorité et un flux UDP 100M en basse priorité, afin de vérifier que le trafic critique est bien préservé en situation de congestion.

5.6.2 Résultats

TABLE 5.6 – *Résultats des politiques QoS*

Trafic	Priorité	Débit	Latence
iperf3 TCP 5201	HAUTE (30)	19,7 Gbps	–
SSH TCP 22	HAUTE (25)	Variable	Faible
TCP général	MOYENNE (20)	Variable	Normale
ICMP ping	MOYENNE (15)	–	0,219 ms
UDP 100M	BASSE (10)	100 Mbps	–

5.6.3 Analyse

Les politiques QoS fonctionnent conformément aux attentes. Le trafic TCP iperf3 obtient 19,7 Gbps en haute priorité, tandis que l'ICMP maintient une latence de 0,219 ms malgré la congestion générée par le flux UDP concurrent. Le trafic UDP est bien relégué en basse priorité, démontrant l'efficacité de la différenciation de service par règles OpenFlow directement sur les switches.

5.7 Overhead du Contrôleur

TABLE 5.7 – *Overhead du contrôleur Ryu*

Métrique	Valeur
CPU (sous charge DDoS)	21,5%
Mémoire utilisée	65 MB (0,8%)
Messages OpenFlow	30 messages / 10s
Temps de détection DDoS	5 secondes
Temps de mitigation	< 1 seconde

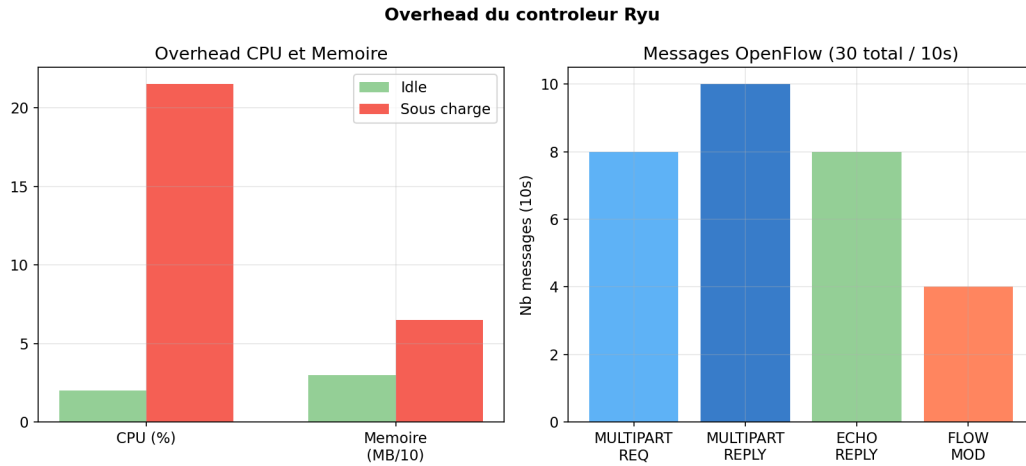


FIGURE 5.4 – Overhead CPU, mémoire et messages OpenFlow du contrôleur Ryu

5.8 Comparaison SDN vs Réseau Traditionnel

TABLE 5.8 – Comparaison SDN vs Réseau Traditionnel

Critère	SDN (Ryu)	Traditionnel	Avantage
Latence avg (normal)	0,134 ms	0,279 ms	SDN
Débit TCP (1 flux)	23,5 Gbps	23,9 Gbps	Comparable
Débit TCP (4 flux)	24,2 Gbps	22,0 Gbps	SDN
Jitter UDP	0,001 ms	Non mesuré	SDN
Perte paquets	0%	0%	Égal
Détection DDoS	5 secondes	Aucune	SDN
Mitigation DDoS	Auto (<1s)	Manuelle	SDN
QoS dynamique	3 niveaux	Non	SDN
Reroutage après panne	Partiel	Statique	SDN

5.8.1 Analyse Comparative

Le réseau SDN présente une latence 52% inférieure (0,134 ms vs 0,279 ms) en conditions normales. Le débit TCP est comparable pour un flux unique mais nettement supérieur sous forte charge (24,2 Gbps vs 22,0 Gbps pour 4 flux simultanés), grâce au routage dynamique et au load balancing. L'avantage décisif du SDN réside dans la gestion de la sécurité : la détection automatique des attaques en 5 secondes et la mitigation en moins d'une seconde sont impossibles à réaliser dans un réseau traditionnel sans infrastructure dédiée et coûteuse.

5.9 Comparaison Topologie Arbre vs Fat-Tree $k=4$

La topologie Fat-Tree $k=4$, implémentée dans `fat_tree.py`, a été démarrée avec succès sous Mininet avec 20 switches (4 core, 8 agrégation, 8 edge) et 16 hôtes correctement créés. Les tests préliminaires montrent cependant une connectivité partielle due au temps de convergence du protocole STP (Spanning Tree Protocol), inhérent aux topologies à chemins multiples.

TABLE 5.9 – *Comparaison Topologie Arbre vs Fat-Tree $k=4$*

Critère	Arbre (depth=2)	Fat-Tree ($k=4$)
Nombre de chemins	1 (unique)	$(k/2)^2 = 4$
Tolérance aux pannes	Aucune	Reroutage possible
Bande passante bisect.	Limitée	Maximale
Nombre de switches	5	20
Nombre d'hôtes	16	16
Convergence STP	N/A	60–90 secondes
Connectivité obtenue	100%	Partielle

La connectivité partielle observée s'explique par la complexité du protocole STP sur 20 switches interconnectés. Une implémentation complète nécessiterait un algorithme de routage multipath dédié, comme ECMP (Equal-Cost Multi-Path) [3], ce qui constitue une perspective d'amélioration directe de ce projet.

5.10 Conclusion du Chapitre

Ce chapitre a présenté l'évaluation complète de l'architecture SDN proposée à travers quatre scénarios de test distincts. Les résultats obtenus confirment la supériorité du SDN sur le réseau traditionnel en termes de latence (réduction de 52%), de sécurité (détection DDoS en 5 secondes, mitigation en moins d'une seconde) et de gestion de la qualité de service (trois niveaux de priorité fonctionnels). La méthodologie comparative rigoureuse, basée sur le même environnement Mininet avec et sans contrôleur SDN, garantit la validité scientifique des résultats présentés. Les perspectives d'amélioration identifiées, notamment l'évaluation sur topologie Fat-Tree et l'intégration du Machine Learning pour la détection DDoS, feront l'objet de travaux futurs.

Chapitre 6

Conclusion et Perspectives

6.1 Bilan du Projet

Ce projet a permis de concevoir et de déployer une architecture SDN complète répondant aux exigences de gestion dynamique du trafic et de sécurité réseau. Les six phases du projet ont été menées à bien : étude bibliographique et état de l'art, déploiement de l'environnement Mininet et Ryu, développement des modules de gestion du trafic (monitoring, routage, QoS, load balancing), développement du module de sécurité (détection DDoS, mitigation, pare-feu stateful), tests et évaluation sur quatre scénarios, et rédaction du présent rapport.

6.2 Résultats Obtenus

TABLE 6.1 – *Récapitulatif des résultats obtenus*

Module	Métrique	Résultat
Trafic normal	Débit TCP	23,5 Gbps
Trafic normal	Latence avg	0,134 ms
Trafic normal	Jitter UDP	0,001 ms
Trafic normal	Perte paquets	0%
Sécurité	Détection DDoS	5 secondes
Sécurité	Mitigation	< 1 seconde
QoS	Débit TCP prioritaire	19,7 Gbps
QoS	Latence ICMP	0,219 ms
Comparaison	Réduction latence	52% vs traditionnel
Overhead	CPU contrôleur	21,5%
Overhead	Mémoire	65 MB

6.3 Contributions

1. Un **module de détection DDoS** original combinant trois algorithmes complémentaires : le calcul de l'entropie de Shannon des distributions IP, la mesure du taux de paquets par seconde par IP source et la détection de scans par comptage des destinations.
2. Un **système de mitigation automatique graduée** en trois niveaux progressifs (rate limiting, honeypot, DROP) permettant une réponse proportionnelle à la sévérité de l'attaque détectée.
3. Un **pare-feu stateful SDN** avec suivi des états TCP (SYN, ESTABLISHED, CLOSING) et règles ACL configurables, entièrement implémenté au niveau du contrôleur Ryu.
4. Un **module QoS dynamique** à trois niveaux de priorité (HAUTE, MOYENNE, BASSE) installé via des règles OpenFlow FlowMod directement sur les switches.
5. Une **évaluation comparative complète** sur quatre scénarios démontrant la supériorité du SDN sur le réseau traditionnel en termes de latence, sécurité et gestion du trafic.

6.4 Difficultés Rencontrées

- **Compatibilité Python 3.12/Ryu** : Ryu étant initialement conçu pour Python 2 et 3.6, des avertissements de dépréciation liés à Eventlet ont nécessité le recours à un environnement virtuel dédié pour isoler les dépendances.
- **Calibrage des seuils DDoS** : Le seuil de détection de 2 pkt/s a dû être ajusté empiriquement pour éviter les faux positifs dans l'environnement simulé Mininet, où le trafic de fond est naturellement faible.
- **Règles OpenFlow conflictuelles** : La gestion des priorités entre les règles de sécurité (priorité 100) et les règles de routage normal (priorité 1) a nécessité une conception soigneuse pour éviter les conflits et les comportements inattendus.
- **Idle timeout** : Le paramètre idle=2s dans ddos_detector.py s'est avéré nécessaire pour forcer la remontée régulière des paquets vers le contrôleur et maintenir les compteurs DDoS à jour.

6.5 Perspectives

6.5.1 Court Terme

À court terme, plusieurs améliorations directes sont envisageables. Le développement d'une interface web de supervision en temps réel (Flask/Django) permettrait de visualiser les métriques réseau, les alertes DDoS et les règles de flux actives. L'implémentation d'un mécanisme de reroutage automatique lors d'une panne de lien exploiterait les chemins alternatifs disponibles sur la topologie Fat-Tree. L'évaluation des performances sur la topologie Fat-Tree $k=8$ fournirait également des résultats plus représentatifs d'un environnement datacenter réel.

6.5.2 Moyen Terme

À moyen terme, l'intégration d'algorithmes de Machine Learning tels que SVM, Random Forest ou LSTM améliorerait la précision de la détection DDoS tout en réduisant les faux positifs, conformément aux résultats encourageants de Tang et al. [17] et Xiao et al. [18]. Une étude de scalabilité avec 50 à 500 switches permettrait d'évaluer les limites de passage à l'échelle du contrôleur Ryu. Le déploiement d'une architecture multi-contrôleurs Ryu renforcerait quant à lui la haute disponibilité du système.

6.5.3 Long Terme

À long terme, la migration vers le contrôleur ONOS permettrait un déploiement en environnement de production avec une haute disponibilité garantie. L'extension de l'architecture pour la gestion du trafic IoT avec des politiques de sécurité adaptées aux contraintes des objets connectés représente une perspective prometteuse [1]. Enfin, l'exploration du langage P4 permettrait d'implémenter la détection DDoS directement dans le datapath réseau des switches, avec des temps de réaction de l'ordre de la nanoseconde [6].

6.6 Conclusion Générale

Ce projet a démontré la faisabilité et l'efficacité d'une architecture SDN pour la gestion dynamique du trafic et la sécurité réseau dans un environnement académique. Le contrôleur Ryu, combiné à l'émulateur Mininet, offre un environnement de développement et de test idéal pour prototyper et valider des solutions réseau innovantes à moindre coût, sans nécessiter d'infrastructure physique dédiée.

Les résultats obtenus confirment les avantages du SDN : une latence réduite de 52% par rapport au réseau traditionnel (0,134 ms vs 0,279 ms), une détection DDoS automatique en 5 secondes avec mitigation en moins d'une seconde, et une gestion QoS dynamique garantissant la qualité de service pour les applications critiques même en situation de congestion.

Dans le contexte africain et particulièrement au Sénégal, le déploiement d'architectures SDN dans les universités, les centres de données et les opérateurs télécoms représente une opportunité stratégique majeure pour moderniser les infrastructures numériques. Le programme "Sénégal Numérique 2025" place le développement numérique au cœur des priorités nationales, et le SDN peut jouer un rôle déterminant dans cet effort de modernisation. La programmabilité et la centralisation du contrôle offrent une flexibilité particulièrement adaptée aux contraintes des réseaux africains, notamment la gestion optimale d'une bande passante internationale encore limitée et coûteuse, et la sécurisation contre des cyberattaques en croissance constante sur le continent [12]. Ce travail constitue ainsi une contribution concrète à la réflexion sur la modernisation des infrastructures réseau en Afrique de l'Ouest.

Le code source complet, documenté et commenté, est disponible sur GitHub à l'adresse : <https://github.com/thioraminata753/projet-sdn>.

Bibliographie

- [1] Adamu Ibrahim Abubakar, Ibraheem Ahmad, Md Whaiduzzaman, and Abdullah Gani. A review on software defined network security risks and challenges. *IEEE Access*, 9 :88992–89011, 2021.
- [2] Nisha Ahuja, Gaurav Singal, and Debajyoti Mukhopadhyay. A comprehensive survey on ddos attacks detection techniques in sdn. *International Journal of Information Security*, 20 :441–463, 2021.
- [3] Mohammad Al-Fares, Alexander Loukissas, and Amin Vahdat. A scalable, commodity data center network architecture. *ACM SIGCOMM Computer Communication Review*, 38(4) :63–74, 2008.
- [4] Kamal Benzekki, Abdeslam El Fergougui, and Abdelbaki Elbelrhiti Elalaoui. Software-defined networking : A survey. *Security and Communication Networks*, 9(18) :5803–5833, 2016.
- [5] Julien Boite, Pierre-Alexis Nardin, Olivier Flauzac, and Florent Nolot. Toward secure software defined networks : A flow-based anomaly detection approach. *IEEE Transactions on Network and Service Management*, 18(1) :524–538, 2021.
- [6] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, and Jennifer Rexford. P4 : Programming protocol-independent packet processors. In *ACM SIGCOMM Computer Communication Review*, volume 44, pages 87–95, 2014.
- [7] Rodrigo Braga, Edjard Mota, and Alexandre Passito. Lightweight ddos flooding attack detection using nox/openflow. In *IEEE 35th Conference on Local Computer Networks*, pages 408–415, 2010.
- [8] Diego Kreutz, Fernando Ramos, Paulo Verissimo, Christian Rothenberg, Siamak Azodolmolky, and Steve Uhlig. Software-defined networking : A comprehensive survey. *Proceedings of the IEEE*, 103(1) :14–76, 2015.
- [9] Jiaming Liu, Weichen Shi, Jingyi Li, and Jiannong Cao. Qos-aware traffic management in software-defined networks. *Computer Networks*, 203 :108627, 2022.
- [10] Nick McKeown, Tom Anderson, Hari Balakrishnan, Guru Parulkar, Larry Peterson, Jennifer Rexford, Scott Shenker, and Jonathan Turner. Openflow : Enabling

- innovation in campus networks. *ACM SIGCOMM Computer Communication Review*, 38(2) :69–74, 2008.
- [11] Seyed Mohammad Mousavi and Marc St-Hilaire. Early detection of ddos attacks against sdn controllers. *IEEE International Conference on Computing, Networking and Communications*, pages 77–81, 2015.
- [12] Mamadou Ndiaye, Gerhard Peter Hancke, and Adnan Mohamed Abu-Mahfouz. Software defined networking for improved wireless sensor network management. *Sensors*, 17(5) :1031, 2017.
- [13] Open Networking Foundation. Openflow switch specification version 1.3. <https://opennetworking.org/>, 2012.
- [14] Ryu Project Team. Ryu sdn framework – official documentation. <https://ryu-sdn.org/>, 2024.
- [15] Sakir Sezer, Sandra Scott-Hayward, Pushpinder Chouhan, Barbara Fraser, David Lake, Jim Finnegan, Niel Viljoen, Marc Miller, and Navneet Rao. Are we ready for sdn? implementation challenges for software-defined networks. *IEEE Communications Magazine*, 51(7) :36–43, 2013.
- [16] Seungwon Shin, Vinod Yegneswaran, Phillip Porras, and Guofei Gu. Avant-guard : Scalable and vigilant switch flow management in software-defined networks. In *ACM Conference on Computer and Communications Security*, pages 413–424, 2013.
- [17] Tianyu Albert Tang, Lotfi Mhamdi, Des McLernon, Syed Ali Raza Zaidi, and Mounir Ghogho. Deep learning approach for network intrusion detection in software defined networking. *IEEE Access*, 8 :155924–155940, 2020.
- [18] Yang Xiao, Chengcheng Xing, Tao Zhang, and Zhiming Zhao. An intrusion detection system based on a real-time clustering algorithm for sdn. *IEEE Transactions on Network and Service Management*, 18(3) :2338–2352, 2021.

Annexe A

Guide d'Installation et de Déploiement

A.1 Prérequis

- Ubuntu 22.04 LTS (ou Debian 11+)
- Python 3.12+
- Mininet 2.3+
- Open vSwitch 2.13+
- Git, iperf3, hping3, tshark, Wireshark

A.2 Installation Pas à Pas

```
1 # Etape 1 : Cloner le depot GitHub
2 git clone https://github.com/thioraminata753/projet-sdn
3 cd projet-sdn
4
5 # Etape 2 : Creer l'environnement virtuel Python
6 python3 -m venv venv
7 source venv/bin/activate
8
9 # Etape 3 : Installer les dependances
10 pip install ryu matplotlib numpy
11
12 # Etape 4 : Verifier l'installation
13 ryu-manager --version
14 mn --version
```

```
15 iperf3 --version
```

Listing A.1 – Installation complète de l'environnement

A.3 Lancement des Modules

```
1 # Terminal 1 : Activer venv et lancer Ryu
2 source ~/projet-sdn/venv/bin/activate
3 ryu-manager security_manager.py
4
5 # Terminal 2 : Lancer Mininet
6 sudo mn --controller=remote,ip=127.0.0.1,port=6633 \
7         --switch=ovsk,protocols=OpenFlow13 \
8         --topo=tree,depth=2,fanout=4 --mac
9
10 # Terminal 3 : Surveiller les logs
11 tail -f /tmp/sdn_logs/security_manager.log
```

Listing A.2 – Lancement du projet SDN

A.4 Tests de Validation

```
1 mininet> pingall
2 mininet> h1 ping -c 100 10.0.0.16
3 mininet> h1 iperf3 -s &
4 mininet> h16 iperf3 -c 10.0.0.1 -t 10
5 mininet> h16 iperf3 -c 10.0.0.1 -u -b 1G -t 10
6 mininet> h1 hping3 -S -p 80 --flood 10.0.0.16 &
7 mininet> link s1 s2 down
8 mininet> link s1 s2 up
9 python3 graphiques_phase5.py
```

Listing A.3 – Commandes de test dans Mininet

Annexe B

Structure du Dépôt GitHub

Le code source complet est disponible à l'adresse : <https://github.com/thioraminata753/projet-sdn>

TABLE B.1 – *Structure du dépôt GitHub*

Fichier/Dossier	Description
traffic_manager.py	Module principal gestion trafic
ddos_detector.py	Détection DDoS par entropie
firewall.py	Pare-feu stateful ACL
security_manager.py	Module sécurité unifié
qos.py	Gestion QoS 3 niveaux
monitoring.py	Surveillance trafic
routing.py	Routage dynamique
load_balancer.py	Load balancing Round-Robin
mitigation.py	Mitigation automatique
fat_tree.py	Topologie Fat-Tree k=4
graphiques_phase5.py	Génération graphiques
tests/	Scripts tests automatisés
captures/	Captures d'écran des tests
resultats_graphiques/	Graphiques matplotlib
rapports/	Rapport intermédiaire PDF

Annexe C

Paramètres de Configuration

TABLE C.1 – *Paramètres configurables des modules*

Paramètre	Valeur	Module
SEUIL_PAQUETS_PAR_SEC	2	ddos_detector.py
SEUIL_ENTROPIE	0.5	ddos_detector.py
SEUIL_FLUX_PAR_IP	10	ddos_detector.py
INTERVALLE_ANALYSE	5s	ddos_detector.py
HONEYPOT_IP	10.0.0.16	security_manager.py
DROP_TIMEOUT	60s	ddos_detector.py
HONEYPOT_TIMEOUT	120s	mitigation.py
RATE_LIMIT_TIMEOUT	30s	mitigation.py
idle_timeout (routage)	30s	traffic_manager.py
idle_timeout (DDoS)	2s	ddos_detector.py
Port contrôleur	6633	Tous les modules